BÁO CÁO THỰC HÀNH LAB 8
LOCAL DNS ATTACK – DNS SPOOFING & CACHE POISONING

Học phần: Thực hành An toàn Bảo mật Hệ thống thông tin – Năm học 2026–2027

Họ và tên	Hoàng Thanh Trà
Mã số sinh viên	1150080120
Lớp	11_ĐHCNPM_2 (11CNPM2)
Repository	https://github.com/thtra04/LAB_AT_BMHTTT
Link video YouTube	https://youtu.be/CzY6DpWF968
Đề bài	LAB8 – Local DNS Attack (tài liệu ghi LAB 9), thư mục đề/

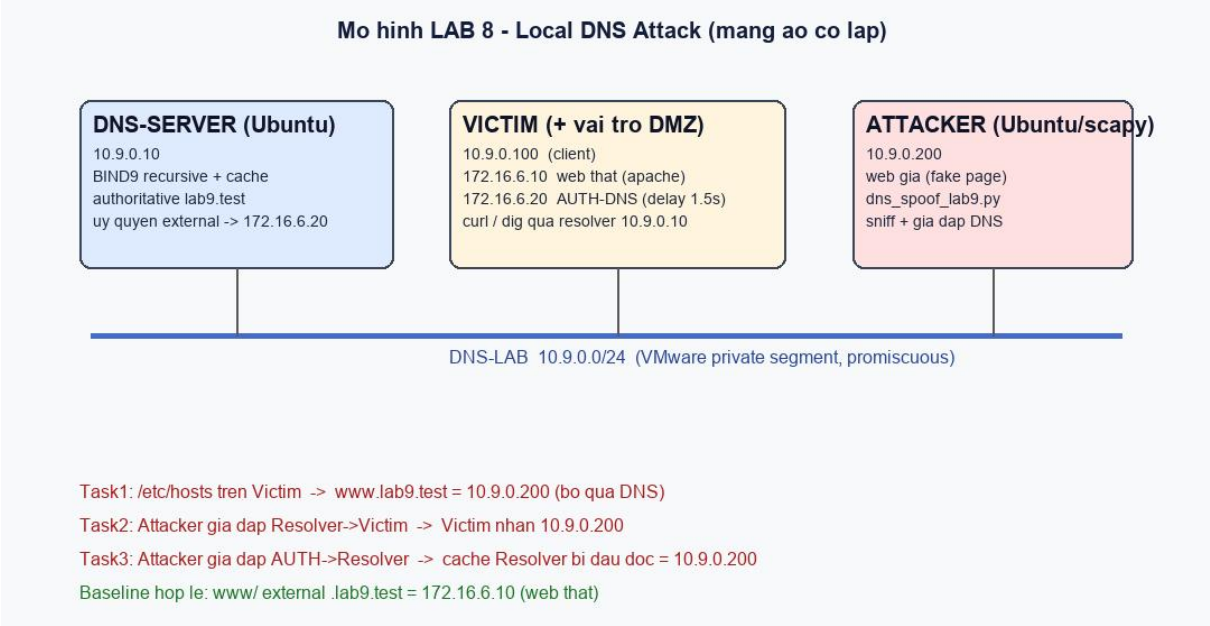
1. Tên bài Lab và mục tiêu

Bài thực hành theo tài liệu Local DNS Attack của SEED Labs (trong repo đặt ở thư mục LAB8; nội dung tài liệu ghi là "LAB 9 – Local DNS Attack"). Mục tiêu dựng một DNS-SERVER BIND9 làm resolver/cache riêng cho một mạng ảo cô lập, quan sát gói DNS, và thực hiện ba kỹ thuật tấn công DNS nội bộ:

1. Task 1 – HOSTS-file redirection: chuyển hướng tên miền ngay trên máy nạn nhân bằng /etc/hosts.
2. Task 2 – Direct DNS spoofing: kẻ tấn công cùng LAN giả mạo phản hồi DNS gửi thẳng tới nạn nhân.
3. Task 3 – DNS cache poisoning: đầu độc cache của resolver, ảnh hưởng mọi client dùng chung resolver.

Sau đó khôi phục cache/cấu hình và phân tích các biện pháp phòng thủ (phân đoạn mạng, kiểm soát DNS egress, DNSSEC/DNS COOKIE, giới hạn promiscuous mode).

2. Mô hình và môi trường thực hiện



Hình 1. Mô hình LAB 8 – ba máy trong mạng ảo cô lập DNS-LAB 10.9.0.0/24

Toàn bộ thí nghiệm chạy trong một mạng ảo riêng, cô lập, chỉ dùng các tên miền .test và các IP đã hard-code trong bộ tài nguyên; không thực hiện trên Internet, Wi-Fi hay mạng thật. Các thành phần:

- DNS-SERVER 10.9.0.10 (Ubuntu): BIND9 vừa authoritative cho zone lab9.test, vừa recursive/cache cho 10.9.0.0/24; tên external.lab9.test được uỷ quyền (delegation) tới AUTH-DNS 172.16.6.20.
- VICTIM 10.9.0.100 (Ubuntu): client gửi truy vấn DNS. Máy này đồng thời đóng vai DMZ: alias 172.16.6.10 chạy web "thật" (Apache) và alias 172.16.6.20 chạy AUTH-DNS mô phỏng (auth_dns_delayed.py, trễ 1.5s).
- ATTACKER 10.9.0.200 (Ubuntu + Scapy): bật promiscuous, chạy web giả và script dns_spoof_lab9.py để nghe/giả mạo DNS.

Ghi chú về điều chỉnh so với đề (được phép, do tận dụng hạ tầng sẵn có): đề viết cho VirtualBox + pfSense kế thừa "LAB 8", còn máy thực tế dùng VMware Workstation nên không có pfSense/VirtualBox của đề. Để bám sát ba nhiệm vụ cốt lõi, mô hình được rút gọn: ba máy đặt chung một phân đoạn mạng riêng (VMware private segment) có bật promiscuous; vai trò DMZ-WEB (172.16.6.10) và APP-TEST/AUTH-DNS (172.16.6.20) được gộp lên máy VICTIM qua IP alias để kẻ tấn công quan sát được truy vấn Resolver→AUTH phục vụ Task 3. Dải địa chỉ 10.9.0.0/24, 172.16.6.10, 172.16.6.20 và toàn bộ cấu hình/script giữ đúng theo

bộ tài nguyên của đề. Máy tấn công dùng Ubuntu + Scapy thay cho Kali (Scapy chạy tương đương cho mục đích bài này).

3. Cấu hình chính

DNS-SERVER – BIND9 (lắng nghe 10.9.0.10, recursion cho 10.9.0.0/24, uỷ quyền external.lab9.test → 172.16.6.20):

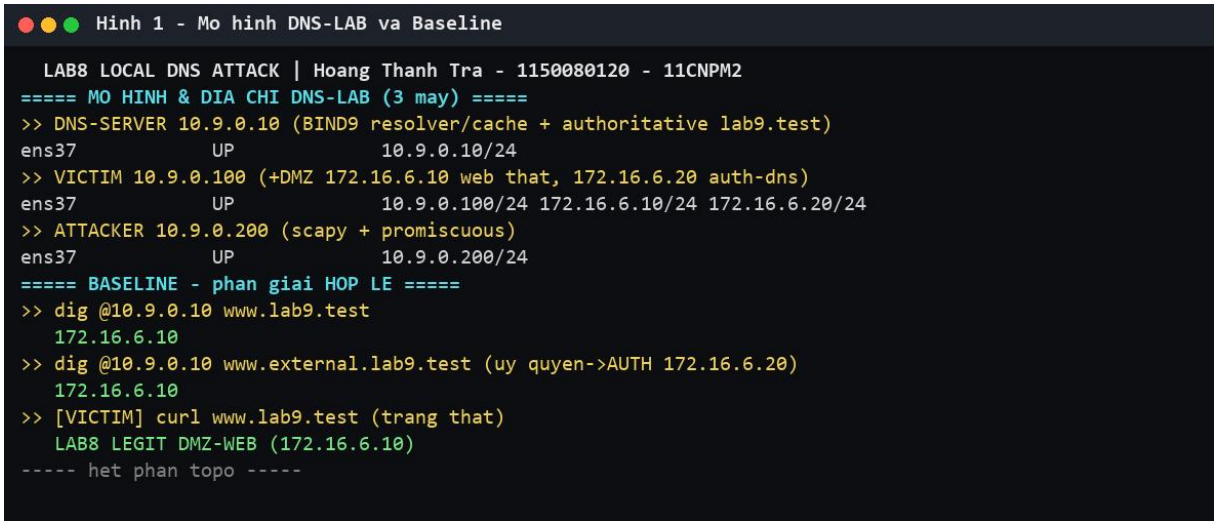
```
zone "lab9.test"  { type master; file "/etc/bind/db.lab9.test"; };
zone "9.10.in-addr.arpa" { type master; file "/etc/bind/db.10.9.0"; };
; db.lab9.test:
www      IN A   172.16.6.10
external IN NS  nsext.lab9.test.
nsext    IN A   172.16.6.20
```

AUTH-DNS mô phỏng (auth_dns_delayed.py) ràng buộc bind 172.16.6.20, trả www.external.lab9.test → 172.16.6.10 và cố ý trễ 1.5s để quan sát rõ "cuộc đua" DNS. Trong named.conf.options tắt DNS COOKIE riêng cho server 172.16.6.20 (chỉ phục vụ tái lập bài on-path, không dùng ngoài thực tế).

ATTACKER – dns_spoof_lab9.py: nghe gói udp/53, lọc đúng tên www.external.lab9.test, sao chép Transaction ID và cổng của truy vấn quan sát được rồi gửi phản hồi giả A=10.9.0.200 (hai chế độ: victim và resolver). Script có whitelist chỉ hoạt động trong 10.9.0.0/24 và 172.16.6.20.

4. Baseline – phân giải hợp lệ trước khi tấn công

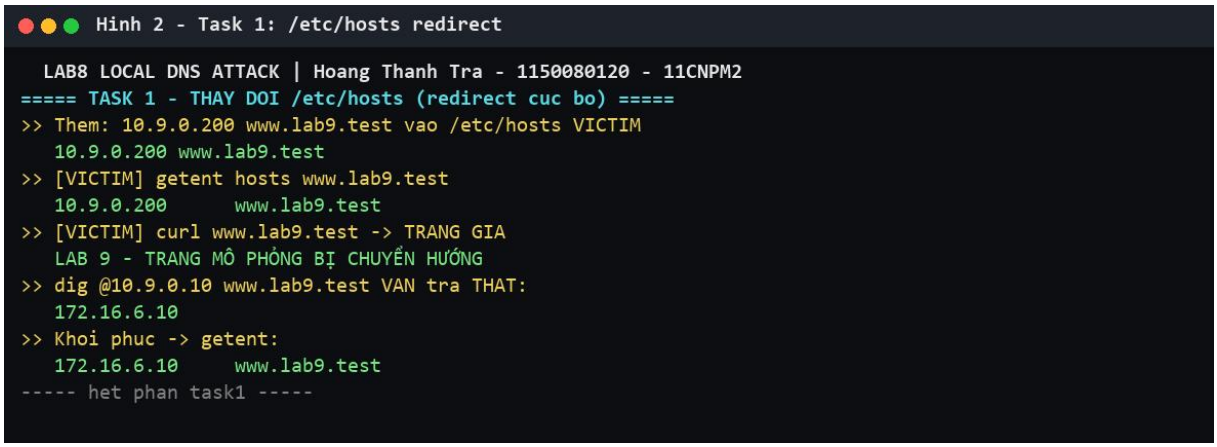
Trước khi tấn công, www.lab9.test và www.external.lab9.test đều phải phân giải về web thật 172.16.6.10.



Hình 2. Địa chỉ ba máy và kết quả baseline (phân giải hợp lệ về 172.16.6.10)

5. Task 1 – Thay đổi /etc/hosts (redirect cục bộ)

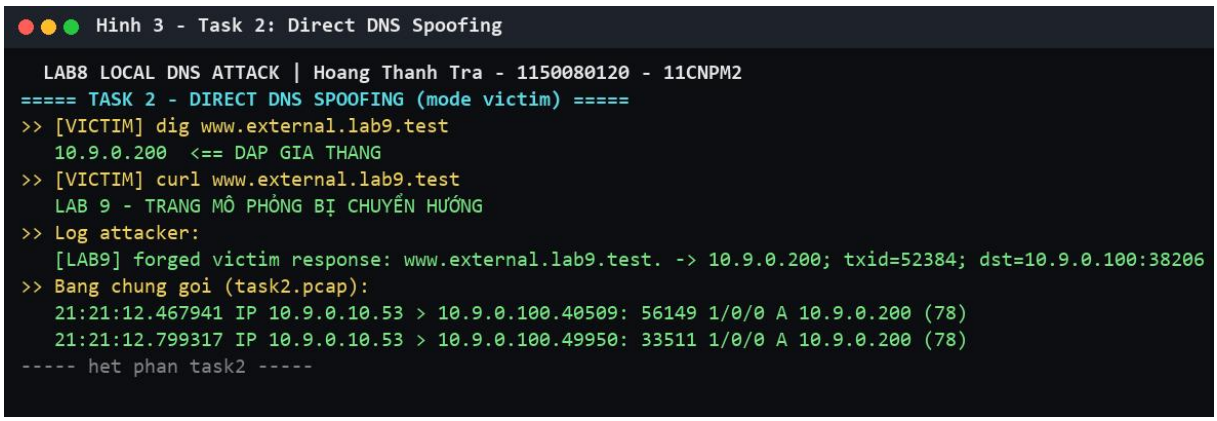
Thêm dòng "10.9.0.200 www.lab9.test" vào /etc/hosts của VICTIM. Các ứng dụng dùng getaddrinfo (getent, curl, trình duyệt) tra /etc/hosts trước nên www.lab9.test bị trỏ về trang giả của attacker; trong khi dig hỏi thẳng DNS-SERVER vẫn trả 172.16.6.10. Sau đó khôi phục /etc/hosts.



Hình 3. Task 1 – /etc/hosts chuyển hướng ứng dụng, dig vẫn trả kết quả thật

6. Task 2 – Direct DNS Spoofing

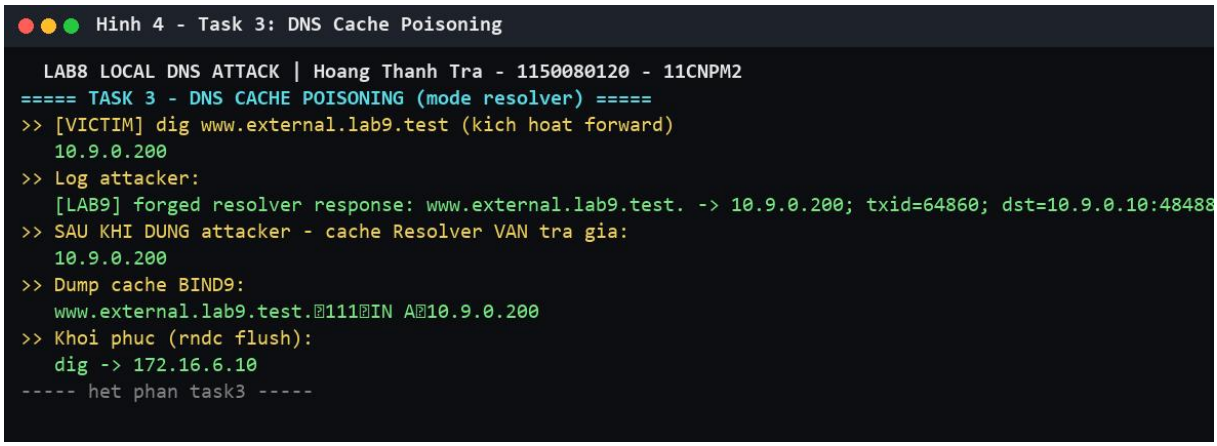
VICTIM chưa sửa /etc/hosts. Attacker cùng mạng quan sát truy vấn Victim→Resolver cho www.external.lab9.test, lập tức gửi một phản hồi DNS giả (giả địa chỉ nguồn 10.9.0.10, đúng Transaction ID và cổng) tới Victim. Do phản hồi hợp lệ bị trễ (AUTH 1.5s), phản hồi giả thắng cuộc đua: Victim nhận 10.9.0.200 và hiển thị trang giả.



Hình 4. Task 2 – phản hồi giả thắng, Victim nhận 10.9.0.200 (kèm gói bắt được)

7. Task 3 – DNS Cache Poisoning

Mục tiêu chuyển sang Resolver. Attacker quan sát truy vấn Resolver→AUTH (172.16.6.20) và gửi phản hồi giả tới Resolver trước phản hồi thật. BIND chấp nhận và lưu bản ghi giả 10.9.0.200 trong cache theo TTL 120s. Sau khi DỪNG attacker, Resolver vẫn trả 10.9.0.200 cho tới khi hết TTL hoặc bị flush – nghĩa là mọi client dùng chung resolver đều bị ảnh hưởng. rndc dumpdb xác nhận bản ghi giả còn trong cache.



Hình 5. Task 3 – cache resolver bị đầu độc (dig và dump cache đều cho 10.9.0.200)

8. Khôi phục môi trường và hardening

- Dừng dns_spoof_lab9.py và web giả trên attacker; flush cache BIND9 (rndc flush) → phân giải trở lại 172.16.6.10.
- Khôi phục /etc/hosts từ bản backup; dừng AUTH-DNS mô phỏng; trả promiscuous về Deny sau buổi thực hành.

Bài tập phòng thủ: bật lại DNS COOKIE mặc định (bỏ send-cookie no cho 172.16.6.20) khiến phản hồi giả on-path khó được chấp nhận hơn ở Task 3; DNS COOKIE chỉ là một lớp bổ sung, không phải cơ chế duy nhất.

9. Trả lời câu hỏi phân tích

Câu 1. Tại sao /etc/hosts có thể ảnh hưởng trình duyệt nhưng dig @DNS-SERVER vẫn trả kết quả DNS gốc?

Trả lời: Các ứng dụng (trình duyệt, curl) phân giải tên qua getaddrinfo/NSS, và theo /etc/nsswitch.conf thì nguồn files (/etc/hosts) được tra trước dns.

dig/nslookup là công cụ truy vấn DNS trực tiếp tới name server, bỏ qua NSS và /etc/hosts, nên vẫn nhận bản ghi gốc từ DNS-SERVER.

Câu 2. Những trường nào của DNS/UDP phải khớp để một phản hồi giả có cơ hội được chấp nhận?

Trả lời: Về UDP: IP nguồn = server hợp lệ, cổng nguồn = 53, cổng đích = cổng nguồn (ephemeral) của truy vấn. Về DNS: Transaction ID trùng, phần Question (qname/qtype/qclass) trùng, và phản hồi giả phải đến trước phản hồi thật. Nếu bật DNS COOKIE thì cookie cũng phải khớp.

Câu 3. Tại sao Task 2 chỉ ảnh hưởng Victim hiện tại, còn Task 3 ảnh hưởng nhiều client dùng cùng resolver?

Trả lời: Task 2 giả đáp trực tiếp cho một phiên truy vấn của Victim, không được lưu lại nên chỉ Victim đó, lần đó bị lừa. Task 3 ghi bản ghi giả vào cache của resolver dùng chung; mọi client hỏi resolver trong thời gian TTL đều nhận cùng kết quả giả.

Câu 4. Vai trò của TTL trong cache poisoning là gì?

Trả lời: TTL của bản ghi giả quyết định thời gian nó tồn tại trong cache. TTL càng lớn, ảnh hưởng càng kéo dài dù attacker đã dừng, cho đến khi hết TTL hoặc cache bị flush.

Câu 5. Tại sao VirtualBox/VMware Promiscuous Mode = Allow VMs cần thiết trong bài này?

Trả lời: Để NIC của attacker nhận được gói unicast giữa Victim↔Resolver và Resolver↔AUTH mà nó không phải đích. Không bật promiscuous, switch ảo chỉ chuyển gói tới đúng cổng đích nên attacker không "nghe" được truy vấn để lấy Transaction ID/cổng mà giả đáp.

Câu 6. Vì sao dùng mạng DNSLAB riêng thay vì tấn công trực tiếp DC01/DNS của LAN?

Trả lời: Để cô lập rủi ro, không phá dịch vụ DNS thật và miền LAN; dễ snapshot/khôi phục; tuân thủ phạm vi an toàn của bài (chỉ .test, IP hard-code), tránh ảnh hưởng ngoài phòng lab.

Câu 7. Khác nhau giữa authoritative DNS, recursive resolver và caching resolver?

Trả lời: Authoritative: nắm dữ liệu gốc của một zone và đưa câu trả lời cuối cùng cho zone đó. Recursive resolver: thay client đi hỏi lần lượt root→TLD→authoritative để tổng hợp kết quả. Caching resolver: recursive có lưu kết quả theo TTL để trả nhanh lần sau. Một BIND có thể kiêm cả ba, như DNS-SERVER trong bài.

Câu 8. DNSSEC giải quyết vấn đề gì và vì sao zone .test unsigned không tự được bảo vệ?

Trả lời: DNSSEC ký số các bản ghi (RRSIG) theo chuỗi tin cậy từ root, cho phép resolver xác thực tính toàn vẹn và nguồn gốc để phát hiện bản ghi bị giả. Zone .test không được ký (unsigned) nên không có chữ ký để kiểm chứng; ngoài ra mạng cô lập không có chuỗi tin cậy từ root, nên DNSSEC không bảo vệ được.

Câu 9. Tại sao chỉ chặn UDP/53 là chưa đủ để kiểm soát DNS hiện đại?

Trả lời: DNS còn chạy trên TCP/53 (câu trả lời lớn, zone transfer), DoT (TCP/853), DoH (HTTPS/443), DNS-over-QUIC. Client có thể né bằng DNS mã hóa hoặc cổng khác. Cần ép client chỉ dùng resolver được chỉ định và chặn mọi đường DNS khác (egress control), thay vì chỉ chặn UDP/53.

Câu 10. Nêu tối thiểu bốn biện pháp hardening sau khi hoàn thành bài.

Trả lời: (1) Phân đoạn mạng – attacker không cùng L2 với client/resolver. (2) DNS egress control – ép client chỉ dùng resolver chỉ định, chặn DNS trực tiếp ra ngoài. (3) Bật DNSSEC validation cho zone có ký, kèm random hoá source-port/Transaction ID và DNS COOKIE. (4) Tắt promiscuous mode và giám sát bất thường (phản hồi DNS trùng lặp, thay đổi cache bất thường); ưu tiên DoT/DoH tới resolver tin cậy.

10. Đối chiếu ảnh minh chứng bắt buộc

- Địa chỉ IP của DNS-SERVER, Victim, Attacker đúng bảng (Hình 2).
- BIND9 named-checkconf/named-checkzone thành công (mục 3, cấu hình đã validate).
- Baseline www/external.lab9.test → 172.16.6.10 (Hình 2).
- Task 1: curl đi tới 10.9.0.200 trong khi dig trực tiếp vẫn trả 172.16.6.10 (Hình 3).
- Task 2: có truy vấn + phản hồi giả; Victim nhận 10.9.0.200 (Hình 4, kèm gói bắt được).

- Task 3: phản hồi giả tới resolver và cache chứa 10.9.0.200 (Hình 5, dump cache).
- Sau flush: kết quả trở về 172.16.6.10 (mục 8).
- Promiscuous Mode trả về Deny sau lab (mục 8).

11. Kết luận

Bài thực hành đã dựng thành công mạng DNS-LAB cô lập và minh hoạ đầy đủ ba kỹ thuật Local DNS Attack: chuyển hướng bằng /etc/hosts, giả mạo DNS trực tiếp tới nạn nhân, và đầu độc cache resolver (ảnh hưởng nhiều client trong suốt TTL). Qua đó thấy rõ DNS không ký là mục tiêu dễ bị giả mạo khi kẻ tấn công ở cùng đường truyền, và các lớp phòng thủ cần kết hợp: phân đoạn mạng, kiểm soát DNS egress, DNSSEC/DNS COOKIE, random hoá và giám sát. Toàn bộ thí nghiệm thực hiện trong môi trường ảo cô lập với tên miền .test, và đã khôi phục sạch sau khi hoàn tất.